

BYTES ARE NOT LATENCY: ALLOCATED SIZE, MEASURED TRAFFIC, DECODE SPEED, AND QUALITY OF KV-CACHE QUANTIZATION ON A SINGLE BLACKWELL WORKSTATION GPU

Anonymous Authors¹

A REPRODUCIBILITY

As in the main paper, TurboQuant[†] denotes TurboQuant as ported, with split-KV count 4 (Section 2.2). The results were produced at repository commit [anonymized]. The study used model `meta-llama/Llama-3.1-8B-Instruct` at revision `0e9e39f249a16976918f6564b8830bc894c89659`. Performance runs used measurement container [anonymized], and quality runs used its derivative [anonymized]. Performance data are identified by the tag [anonymized]. The hardware manifest, per-run configuration, raw samples, and five independent process measurements per point are retained under content-addressed roots with indefinite retention: full-scan wall-clock root [anonymized], profiler root [anonymized], modeling root [anonymized], and joint-results root [anonymized]. The KIVI and KVQuant source patches are stored in the repository with their manifests. Two CPU-only packages run without a GPU or network access: the modeling package (Python standard library only) regenerates the modeling audit, figures, and predictor examples, and the joint-results package (Python with `pyarrow`) regenerates the per-stage outcome table, the joined quality–performance table, and the empty table of qualified configurations byte-for-byte. The joint-results package also carries the frozen inputs of Tables 2–4 and Figure 1a; the frozen columns of Table 3 computed from it match the table digit for digit. The v1.0 release notes and manuscript use the numbering of an earlier version of this paper; Table A1 maps it to this version.

Post-hoc analyses. The analyses of Appendices C, D, and F were defined after the results were known and read only the frozen roots above; none reruns a timing scan or a quality run. Their code and outputs are sealed under the same retention scheme: traffic model, ceilings, bandwidth, elasticity, capacity, and decomposition [anonymized]; kernel breakdowns from the existing traces [anonymized]; the Nsight

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the Machine Learning and Systems (MLSys) Conference. Do not distribute.

Table A1. Table, figure, and section numbers in the v1.0 release (manuscript and release notes) and in this version.

v1.0	This version
Table 5 (candidate scores)	Table A5
Table 6 (model D errors)	Table A6
Table 7 (perplexity)	Table 5; KVQuant rows in Table A7
Table 8 (LongBench v2)	Table 6
Table 9 (joint outcome)	Table 7
Figure 2 (eager/Graph ratio)	Figure A1
Figure 3 (held-out predictions)	Figure A2
Sections 4.1–4.3 (modeling)	Section 4 and Appendix E
Section 6.2 (reproducibility)	Appendix A
Section 6.3 (conclusions)	Section 6.2
Tables 1–4, Figure 1, Sections 1–3 and 5	unchanged; Figure 1 adds the post-hoc panel b

diagnostics of Appendix D, raw [anonymized] and analysis [anonymized]; the KVQuant quality exclusion record [anonymized]; the final-revision analyses (cache-only ceiling, KIVI kernel breakdown) [anonymized]; and KIVI’s adjusted ratio S_{adj} [anonymized]. The addendum runs of Appendix G (amendment, code, per-process records, and raw files) are sealed under [anonymized], with a CPU-only script that recomputes their derived numbers. The post-hoc code is on the repository’s main branch, not in the v1.0 release; rerunning its CPU analyses on the retained roots reproduces their sealed tables and figures unchanged. Every number that the v1.0 packages do not contain, including the post-hoc columns of Table 3 and panel b of Figure 1, comes from these artifacts.

Modeling. Beyond the limits in Appendix E, fold-level coefficient vectors were not saved, so held-out predictions are tied to the stored out-of-fold table and the fitting code.

Run accounting. The full scan comprises 2,670 planned runs (534 points $\times$ 5 processes): 2,205 completed runs and 465 predicted capacity-infeasible. Thirty-eight runs were replaced after infrastructure failures, each linked to the run it replaces, and no valid, slow, or unstable observation was rerun; output checksums, kernel paths, and allocations

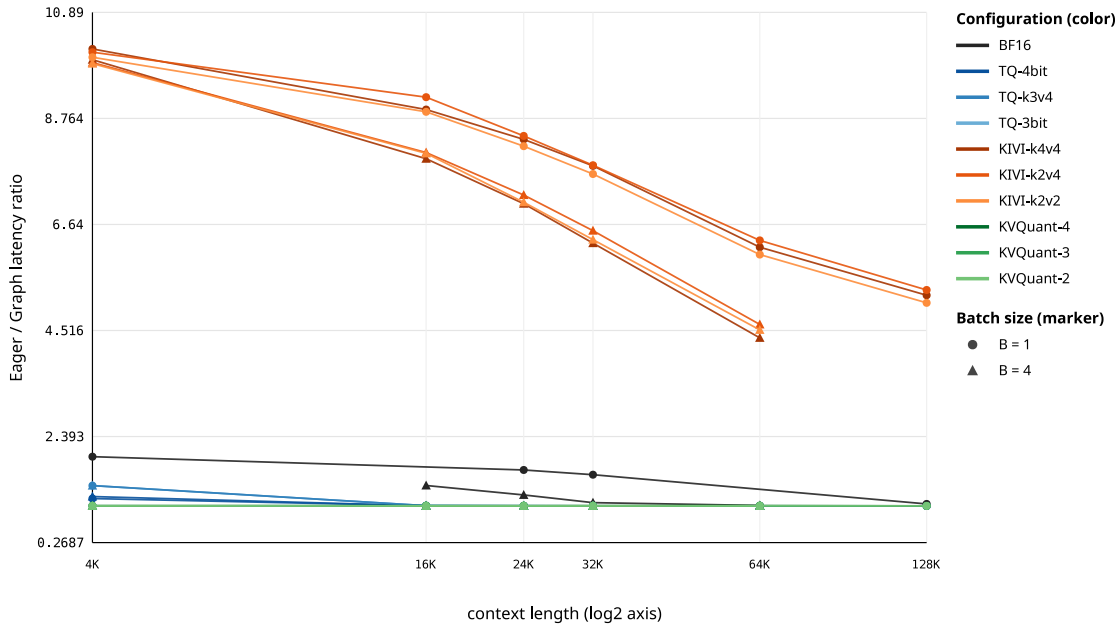


Figure A1. Eager-to-Graph latency ratio versus context for each configuration at $B = 1$ and $B = 4$ (within-configuration mechanism experiment). KIVI's ratio starts near 10 and declines with context. BF16 starts near 2. TurboQuant[†] and KVQuant stay close to 1.

agree across processes. In the eager-versus-Graph experiment, 10 of 120 conditions were predicted infeasible and 105 were stable; five eager conditions (three BF16, two TQ-3bit) exceeded the 3% CV threshold and were kept as unstable, excluded from fitted comparisons, and not rerun. The quality–performance join matches runs by exact configuration identity and retains all 2,670 planned runs, with no unmatched or duplicated identities.

Artifact availability. The code, configurations, and method patches are released at [anonymized URL] as release tag v1.0 (default branch) under the Apache-2.0 license; vendored and patched third-party code keeps its upstream terms, listed in the repository's NOTICE file. The protocol, decision records, and full research history are on the main branch at commit [anonymized]. The two CPU reproduction packages are released as GitHub Release assets. The raw measurements are available from the authors on request. Model weights are gated under the Llama 3.1 Community License and are not redistributed.

Modeling package (Python standard library only), run from the package root:

```
python3 reproduce.py reproduce --package . \
  --output <output_dir>
```

Joint quality–performance package (Python with pyarrow), run from the package root:

```
python3 reproduce.py --reproduce . \
```

```
--output <output_dir>
```

In both cases <output_dir> must not exist yet.

B PROTOCOL TIMELINE

Date (2026)	Event
Jul 22	Scope, model, and performance measurement protocol fixed; post-performance quality protocol (stages, gates, margins) preregistered
Jul 25–31	Measurement container fixed; per-method and unified validation gates passed
Aug 22–26	Pilot scan; knee-densification design (84 adaptive contexts) frozen and executed; an earlier blocked pilot attempt contributed no timing
Aug 26–30	Eager/Graph mechanism experiment; profiler subset
Aug 31–Sep 16	Full scan executed; wall-clock primary analysis completed; an earlier stopped scan attempt contributed no timing
Sep 16–17	Modeling and CPU reproduction package
Sep 17–18	Performance data frozen; exact quality inputs (dataset revisions, sample IDs, prompt and chat-template hashes) and mechanical choices (bootstrap size and resampling units, the bounded C4 frame, length buckets, the CoT subset and budgets) fixed
Sep 19	Correctness stage: all ten configurations fail cross-batch invariance; BF16 diagnosis
Sep 19	Physical- $B = 1$ amendment approved, before any perplexity score
Sep 19–21	Fast perplexity
Sep 21–27	Full perplexity (KIVI-k4v4)
Sep 27–Oct 2	LongBench-E (KIVI-k4v4)
Oct 2–4	LongBench v2 no-CoT (KIVI-k4v4)
Oct 4	Joint validation and scoped quality–performance join
Oct 4–5	Post-hoc review analyses on the frozen data and Nsight diagnostics (Appendices C, D, and F); no timing scan or quality run
Oct 5–6	Post-hoc addendum, amendment committed before any run: TurboQuant with 32 splits, KIVI with a grouped residual (gate failed; diagnostic), vLLM FP8 positive control, synthetic-cache throughput (Appendix G); no quality run

The first full-scan summary computed ratios from CUDA-event medians. A later append-only analysis re-derived the preregistered wall-clock statistics from the same raw files without rerunning any timing, and all timing numbers in this paper use wall-clock time.

C TRAFFIC MODEL, CEILINGS, AND EFFECTIVE BANDWIDTH (POST HOC)

Everything in this appendix was defined after the results were known. It uses the frozen wall-clock process medians, the byte accounting of Section 2.5, and the Nsight Compute profiles, which flush caches before every kernel replay and lock base clocks, so their DRAM bytes are cold-cache counts.

Traffic model. For configuration m at (B, L) we model

Table A2. KVQuant at $B = 1$, 128K: frozen classification (Table 3) $\rightarrow$ key-score and sparse-key kernels counted in the cache path (post hoc). Bandwidth as in Table 3, as a share of peak.

	Cache-path DRAM (GB)	r_{DRAM}	A	Cache-path BW (%)
KVQuant-4	4.54 $\rightarrow$ 8.08	3.82 $\rightarrow$ 2.15	0.82 $\rightarrow$ 1.46	2.0 $\rightarrow$ 3.6
KVQuant-3	3.78 $\rightarrow$ 6.78	4.60 $\rightarrow$ 2.56	0.88 $\rightarrow$ 1.58	1.7 $\rightarrow$ 3.0
KVQuant-2	3.16 $\rightarrow$ 5.62	5.50 $\rightarrow$ 3.09	1.01 $\rightarrow$ 1.79	1.4 $\rightarrow$ 2.4

the DRAM traffic of one decode step as $D_m(B, L) = N_f(B, L) + \alpha_m C_m(B, L)$, where C_m is the allocated cache and α_m is the cache-path DRAM per allocated byte at the profiled point ($B = 1$, 128K): 1.01 for BF16, 0.77–0.81 for TurboQuant, 1.13–1.21 for KIVI, and 1.48–1.81 for KVQuant, whose key-path kernels the model counts as cache traffic (below). The non-cache term $N_f = n_0 + n_1 B(L + 1)$ is fit per family to every $B = 1$ profile of the family: $n_0 = 15.1$ GB and $n_1 \approx 0$ for BF16 and TurboQuant, and $n_0 = 15.3$ GB with $n_1 = 12.6$ and 12.1 KB per sequence token for KIVI and KVQuant, whose kernels write logits-sized intermediates; extending n_1 to $B > 1$ assumes this work is per sequence. At the twelve profiled points other than 128K, the cache term, which is out of sample there, is within 2.8% of the measurement and the total within 0.5%. Post-hoc Nsight Compute profiles of BF16, TQ-4bit, and KIVI-k4v4 at $B = 8$, 32K and $B = 16$, 16K test the extension to larger batches: the modeled total is within 3.1% of the measurement and the non-cache term within 2.6%; the cache term is within 1.2% for BF16 and TQ-4bit, and the model underestimates KIVI’s cache-path traffic by 4.1–4.6%.

KVQuant classification. The frozen classification assigns kernel names containing “rope” to other model work, which put KVQuant’s key-score kernels (VecQuant [2–4] Mat MulKernelNUQPerChannelTransposedRopeMHA BatchedFusedOpt, which read the packed key cache) and its sparse-key kernel (SPMV_ATOMIC_ROPE_BALANCED) outside the cache path. Moving them in adds 3.53, 3.00, and 2.46 GB at 128K for KVQuant-4, -3, and -2 (Table A2) and makes the family’s non-cache fit consistent across bit widths (largest residual 0.8% of total traffic, against 3.0% before). No other family changes. Table 3 keeps the frozen values; the traffic model, the ceilings, the elasticities, and the decomposition use the reclassified KVQuant traffic.

Ceilings. $S_{\text{eq}} = D_{\text{BF16}}/D_m$ uses the as-ported traffic. The ideal cache traffic $D_{\text{ideal,cache},m} = \alpha_{\text{BF16}} C_m$ assumes the method reads its allocation as efficiently as BF16 reads its own ($A = 1$). S_{cache} (Section 3.2) keeps BF16’s mea-

sured non-cache time, $T_{\text{BF16}} - D_{\text{cache,BF16}}/BW_{\text{eff,BF16}}$ with $BW_{\text{eff,BF16}}$ at the same batch size from Table A3 (at least 11.3 ms at every point), and moves $D_{\text{ideal,cache},m}$ at $BW_{\text{peak}} = 1,792$ GB/s, NVIDIA’s published figure, which the Nsight Compute device attributes reproduce (14.001 GHz memory clock $\times 2 \times 512$ -bit bus). Excluding decode workspace and padding from C_m raises TurboQuant’s S_{cache} from 1.02–1.70 to 1.02–1.88 and changes the others by at most 0.01. For comparison, the roofline ceiling $S_{\text{roof}} = T_{\text{BF16}} \cdot BW_{\text{peak}} / (N_{\text{BF16}} + D_{\text{ideal,cache},m})$, 1.37–2.78 at the same-work points, also lets the non-cache work, mostly the 15.1 GB of weights, run at peak bandwidth, so it includes speedups unrelated to the cache: at $B = 1$, 4K it is 1.37–1.39, although the cache path is only 2.9% of BF16’s step time and S_{cache} is 1.02. BF16 itself moves its modeled traffic at 72–85% of peak at the same-work points.

Effective bandwidth. Total: total decode DRAM at 128K over the 128K step time. Cache path: $(D_{\text{cache}}(128K) - D_{\text{cache}}(4K)) / (T(128K) - T(4K))$ at $B = 1$, with $D_{\text{cache}}(4K)$ measured for BF16, TQ-4bit, and KIVI-k4v4 and modeled otherwise. Using $D_{\text{cache}}(128K)$ alone in the numerator gives 92.2% for BF16 and 8.0–12.9% for KIVI.

Elasticity. At each (B, L) where all three configurations of a family were measured, we regress $\log T$ (point medians) on $\log C_{\text{alloc}}$ across the bit widths, and the same regression of the logarithm of modeled traffic gives the slope expected if latency were proportional to traffic. A threshold fixed before any slope was computed keeps the 40 points where modeled cache traffic is at least 25% of the total (92 fall below it; 30 lack a configuration). Pooled within-point slopes are 0.142 [0.140, 0.148] for KIVI (expected 0.304), $-0.057 [-0.057, -0.055]$ for KVQuant (0.242), and 1.243 [1.228, 1.264] for TurboQuant[†] (0.553); the intervals come from 2,000 bootstrap draws over the five process sessions and reflect timing noise only. TurboQuant’s bit widths also select different kernel specializations, so its slope mixes byte and kernel effects.

Steady-state capacity. Steady-state memory is the weights, the allocated cache including its persistent workspace, the endpoint workspace, and the CUDA Graph pool, all from the preregistered feasibility formula; the transient is the prefix-construction peak (MLP intermediates of the unchunked prefill), the prefix control tensors, and KVQuant’s prefix-packing scratch. All values are predictions. Steady state admits 505 of the 534 grid points (end to end: 441), and no end-to-end-feasible point is infeasible in steady state.

D PROFILER DIAGNOSTICS (POST HOC)

TurboQuant geometry and upstream parity. Nsight Systems traces (Graph mode, eight replays) and Nsight Compute reports show the stage-1 grid $(B, 32, 4)$ with 32 threads

Table A3. Effective cache-path bandwidth BW_{eff} (GB/s) from the decomposition $T = c_0 + D_{\text{cache}}/BW_{\text{eff}}$ fitted across contexts at each batch size (Section 4; post hoc).

Configuration	$B = 1$	$B = 2$	$B = 4$	$B = 8$	$B = 16$
BF16	1602	1639	1649	1653	1665
TQ-4bit [†]	14	28	58	113	208
TQ-k3v4 [†]	19	39	78	146	212
TQ-3bit [†]	14	29	60	114	177
KIVI-k4v4	229	342	455	478	477
KIVI-k2v4	199	305	419	443	443
KIVI-k2v2	142	229	321	379	379
KVQuant-4	70	66	78	103	60
KVQuant-3	57	54	64	91	50
KVQuant-2	47	44	52	53	35

per CTA for all three configurations. At $B = 1$ its share of traced GPU kernel time is 55.7% at 4K, 90.6% at 32K, and 97.2% at 128K (TQ-4bit, as ported). Our kernel source matches vLLM v0.25.1 except for imports, and the adapter launches it with the upstream parameters (stage 1: four tokens per tile, one warp, one stage; stage 2: four warps, two stages) except the split count: vLLM’s `tq_max_kv_splits_for_cuda_graph` defaults to 32, while our port, and the G1 reference fixtures, use 4.

Split-count diagnostic. In the measurement container, with the Full Scan execution repository and the same session construction (BF16 restores its prefix snapshot; compressed configurations rebuild the cache from the logical prefix), we traced eight Graph replays per point with the split count at 4 (as ported) and at 32 (Table A4). The 32-split runs swap the split constant and a caller-owned scratch only inside each decode call; in all 33 traced runs, the 12 with 32 splits included, the historical cache and its pointers were unchanged, the adapter fingerprint validated, and outputs were finite (worker checks sealed with the analysis, root [anonymized]). These checks cover finiteness and cache integrity only; the addendum later compared the two paths numerically and in wall-clock time (Appendix G). $B = 8$ at 128K is infeasible, so $B = 8$, 48K replaces it. As-ported traced kernel time per step agrees with the frozen wall-clock medians to within 2.5%.

KVQuant selection kernel. `SelectFixedOutliers1024Cap12Kernel` runs with grid and block (1,1,1) at every traced point, twice per layer (keys and values), so 64 launches per sequence and step. At $B = 1$ it takes 136.4, 130.3, 123.2, and 174.5 ms per step at 24K, 48K, 64K, and 128K (1.9–2.7 ms per launch), 80.2%, 72.6%, 67.5%, and 63.3% of traced GPU kernel time; we did not establish why its time varies with context.

Table A4. TurboQuant split-count diagnostic (Nsight Systems, median of eight replays): stage-1 and whole-step GPU kernel time per decode step (ms) with 4 (as ported) and 32 splits, and the 32-split step time over BF16’s at the same point. Profiler observations of GPU kernel time, not wall-clock benchmark timing.

Config	<i>B</i>	<i>L</i>	Stage 1		Step		32/BF16
			4	32	4	32	
TQ-4bit	1	32K	113.4	14.4	124.4	26.4	1.89
TQ-4bit	1	128K	448.1	56.4	459.4	70.0	3.16
TQ-4bit	8	32K	108.9	39.7	122.3	54.5	1.68
TQ-4bit	8	48K	163.1	61.4	177.2	77.9	—
TQ-k3v4	1	32K	76.6	10.4	88.8	22.4	1.60
TQ-k3v4	1	128K	298.9	40.9	309.5	54.1	2.44
TQ-k3v4	8	32K	77.8	42.3	92.0	57.3	1.76
TQ-k3v4	8	48K	115.4	65.7	129.9	82.2	—
TQ-3bit	1	32K	90.5	11.7	102.2	23.6	1.69
TQ-3bit	1	128K	357.4	45.8	369.0	59.2	2.68
TQ-3bit	8	32K	89.7	46.5	103.9	61.5	1.89
TQ-3bit	8	48K	133.2	72.1	147.6	88.7	—

E PREDICTIVE MODELING

The modeling data are the 2,205 wall-clock process medians at the 441 feasible points, weighted equally per point. The selected model, D, is a knee surface fit separately for each family (BF16, TurboQuant, KIVI, KVQuant):

$$T(B, L, r) = \tau(B, r) + s(B, r) \cdot \max\left(\frac{L}{L_{\max}} - \lambda(B, r), 0\right),$$

with $L_{\max} = 131,071$.

Each of τ , s , and λ is a function of standardized log B and log r_{alloc} through the basis $\{1, z_B, z_r, z_B \cdot z_r\}$; τ and s are exponentiated to stay positive and λ lies in $(0.02, 0.98)$, giving 12 coefficients per family. r_{alloc} is computed at each (B, L) from the byte accounting of Section 2.5, so KIVI’s context-dependent ratio enters directly. Fits minimize mean squared log latency with a small ridge penalty. The alternatives are E, a quadratic in the scalar $\log(B \cdot L/r_{\text{alloc}})$, i.e., the byte law; Surface, a quadratic in $\log B$, $\log L$, and $\log r$; F_{shape} , D plus byte-composition fractions; and $F_{\text{diagnostic}}$, which adds the observed kernel count and is not deployable.

Four holdout protocols were used: leave one batch size out, leave one compressed configuration out within a family (not applicable to BF16), leave the 24K–48K context band out, and run-to-run repeatability (one of the five process sessions held out). The first three define 11 applicable (protocol, family) cells. Among the deployable candidates, the selection rule, fixed in advance, takes the lowest macro mean of cell median relative errors, then the lowest macro mean of cell P95 errors, then fewer coefficients. The targets were median $\leq 5\%$, P95 $\leq 10\%$, same-work sign accuracy $\geq 95\%$, pairwise ranking accuracy $\geq 90\%$, and knee error $\leq 10\%$.

Table A5. Candidate scores. Each value is the mean, over the 11 geometry-holdout cells, of the per-cell median or P95 relative error.

Candidate	Coefficients per family	Macro median rel. error	Macro P95 rel. error	Deployable
E (scalar byte law)	3	34.6%	122.2%	yes
Surface	10	14.2%	56.5%	yes
D (selected)	12	7.9%	164.5%	yes
F_{shape}	24	8.4%	60.8%	yes
$F_{\text{diagnostic}}$	27	7.6%	30.0%	no

Selection uses the same outer holdouts on which we report error, with no nested loop, so the reported scores of the selected model do not account for the selection step.

E is far worse than every alternative on median error, so the scalar byte law we tested does not describe these data. Surface improves on E, and D improves again. F_{shape} has a slightly worse median than D (8.4% vs 7.9%) but a much better P95 (60.8% vs 164.5%); the selection rule ranks by median first and selects D. D has the worst macro P95 of the five (164.5%) because one cell dominates it: BF16 with a batch size held out has a P95 of 1,625.7%, while the other ten cells range from 2.4% to 44.6% (Table A6).

The selected model misses every evaluable target: macro median 7.9% (target 5%), macro P95 164.5% (10%), same-work sign accuracy 93.2% (425/456; 95%), and pairwise ranking accuracy 67.4% (346/513; 90%). Since every measured ratio is below 1, a constant “slower” predictor would score 100%, so sign accuracy is uninformative here. Knee error is not evaluable because no independent knee reference exists.

The BF16 tail is a genuine extrapolation failure. With $B = 1$ held out, D trained on $B \in \{2, 4, 8, 16\}$ predicts 668.6 ms for BF16 at $B = 1$, 4K, where 11.77 ms was measured—a relative error of 5,579%. Over the 15 BF16 edge-extrapolation rows the median error is 21.5% and the P95 4,017%, against 2.5% and 6.0% for the 27 interior-interpolation rows. An audit confirmed units, joins, context scaling, and the point-dependent BF16 ratio; the prediction is reported unmodified and appears as the column of points at the far left of Figure A2. TurboQuant’s latency is not monotone in r (Section 3.3), and holding out one TurboQuant configuration gives the weakest configuration transfer (29.9% median error).

Of 50 configuration–batch curves, 23 have a knee identified within the measured range; 14 are weakly identified, 12 prefer a linear or constant description, and 1 has insufficient feasible span. Intervals come from 1,000 bootstrap draws over the five process sessions. Within the measured domain D interpolates reasonably for these ten configurations: held-

Table A6. Model D held-out relative error, median / P95.

Protocol	BF16	TurboQuant [†]	KIVI	KVQuant
Leave one batch out	3.9% / 1,625.7%	7.8% / 23.1%	8.2% / 20.7%	6.2% / 11.7%
Leave one configuration out	n/a	29.9% / 44.6%	2.9% / 14.1%	4.7% / 16.2%
Leave context band out	1.8% / 2.4%	9.7% / 21.9%	4.9% / 16.2%	7.3% / 13.3%
Run-to-run repeatability (not used for selection)	1.2% / 3.9%	7.9% / 22.0%	3.5% / 11.8%	4.5% / 10.0%

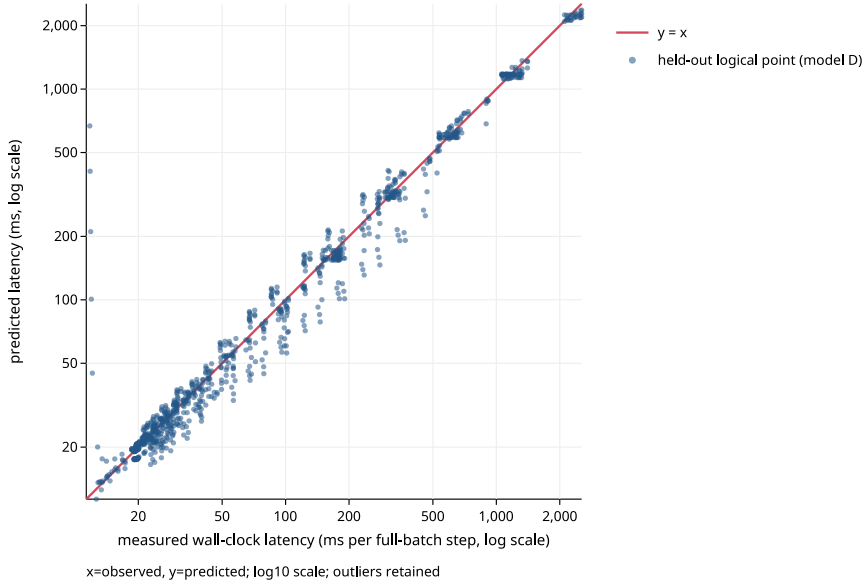


Figure A2. Held-out predicted versus measured latency for model D over the three geometry holdout protocols (968 held-out predictions; both axes in ms on a log scale; red line $y = x$). The vertical column at the far left is the BF16 batch-edge extrapolation failure, which is retained.

out context bands give median errors of 1.8–9.7%, and BF16 interior batch sizes give 2.5% median and 6.0% P95 error. Outside it—batch sizes at the grid edge, quantizers beyond the three measured per family, other models, GPUs, or serving systems—the model is unvalidated, and its 67% ranking accuracy makes it unsuitable for automatic method selection. The evidence favors method-conditioned $B/L/r$ structure over a scalar byte law and leaves open whether B , L , and r are sufficient.

F KVQUANT QUALITY AND EXCLUSION

Perplexity rises about thirtyfold for all three KVQuant configurations, consistently across both datasets and all four lengths in the paired per-anchor data (Table A7). These configurations produce finite outputs, pass cache-dependence controls, and match their method’s reference fixtures, so the degradation is not a crash or numerical overflow. Because it does not shrink from 2 to 4 bits, it is inconsistent with quantization error and points to a defect in the integrated port—the GQA/RoPE compatibility patch, the project-defined outlier capacity, the graph-safe and deterministic kernels, or the

Table A7. KVQuant Fast perplexity versus BF16 (same protocol as Table 5). Intervals are paired 95% CIs on Δ NLL.

Configuration	Fast Δ NLL [95% CI]	Fast rel. PPL	Fast status	Full status
KVQuant-4	3.397 [3.297, 3.500]	+2,888%	FAIL	not run
KVQuant-3	3.387 [3.288, 3.489]	+2,859%	FAIL	not run
KVQuant-2	3.411 [3.319, 3.506]	+2,930%	FAIL	not run

16-window calibration of Section 2.2—whose cause we did not establish; it should not be attributed to KVQuant implementations in general. After the results were known we therefore treat the result as invalid and exclude KVQuant from the quality conclusions. The preregistered FAIL status in Table 7 is unchanged, and the exclusion is recorded with the machine-readable reason `quality_result_invalid_id_suspected_port_defect` (root [anonymized]).

G POST-HOC ADDENDUM MEASUREMENTS

These runs were designed after the main results were known; an amendment fixing their points, configurations, process counts, and interpretation rules was committed before any of them ran, and they change no frozen result. They used the measurement container, GPU, model, and Full Scan worker of the primary scan (CUDA Graph, fixed L , 64 warmup replays, then five timed batches of 256 replays per process, whose median is the process median; host wall-clock), with three independent processes per point instead of five, block-randomized order, and BF16 remeasured in the same run for every ratio. Same-run BF16 medians agree with the frozen ones within 0.25%, and every point is stable ($CV \leq 0.9\%$). Uncertainty is given as the range of the nine ratios between the three BF16 and the three method process medians.

TurboQuant with 32 splits. The stage-1 and stage-2 launches use 32 splits and a caller-owned FP32 scratch of shape $(B, H_q, 32, D+1)$, allocated with the cache before warmup and graph capture; nothing else changes (Table A8). At $B = 1$, 4K, on a WikiText-2 prefix in the eager growing-cache path, the 4- and 32-split paths differ by at most 0.25 in a step-0 logit and agree on 69 of 100 greedy tokens (first divergence at the 39th); Compute Sanitizer memcheck reports no errors. With 32 splits, wall-clock step time over BF16's is 1.57–3.06, against 1.60–3.16 in traced GPU kernel time. At $B = 1$ a two-context fit of $T = c_0 + D_{\text{cache}}/BW_{\text{eff}}$ gives $c_0 = 11.95$ ms (BF16: 11.38) and $BW_{\text{eff}} = 111$ –142 GB/s, 7.0–8.9% of BF16's 1,596 GB/s; D_{cache} is the modeled traffic of Appendix C, whose α was measured on the 4-split kernels.

KIVI with a KV-head-grouped residual (diagnostic only). The adapter computes the residual-window and current-token products as one batched product per operand over $[B \cdot 8, 4, \cdot]$ views of the existing buffers, as the source patch of Section 2.2 does; the quantized-history kernels, softmax, and rollover are unchanged. It passes G1, G3 (with a semantic check in place of G3's literal source-string test for per-head indexing, which the grouped form cannot contain), G4, zero replay allocation, and Compute Sanitizer, but fails the preregistered gate of 100 identical greedy tokens against the existing adapter at $B = 1$, 4K: the first divergence is at the 32nd token for k4v4 and the 82nd for k2v2, the step-0 logits differ by at most 0.13, and the existing adapter is deterministic (100 of 100 against itself). Batching the four query heads of a KV group changes cuBLAS kernel selection and hence FP16 rounding; a variant with four $M = 1$ products per operand also fails (first divergence at the 32nd and 87th token). Its timings are therefore diagnostic only (Table A9). In Nsight Systems at $B = 1$, 4K it issues 1,866 kernels and 12.5 ms of GPU kernel time per step, against 6,826 and 18.3 ms for the existing adapter. Its c_0 is 11.8–12.0 ms (BF16: 11.4–11.7; largest difference 0.46 ms), and its BW_{eff} is 148–

488 GB/s. Against S_{adj} , computed before these runs from the existing decomposition without refitting, the absolute speedup-ratio error is at most 0.026; this compares twelve points and does not validate the $B/L/r$ models or the ceilings. At the two points with $S > 1$ (k2v2, $B = 8$, 24K and 32K), all nine process-pair ratios exceed 1 (1.001–1.004 and 1.004–1.009), from three processes per side.

FP8 positive control. vLLM v0.25.1 ran in a separate environment with the same model and revision, CUDA Graphs on, and prefix caching enabled, which its latency benchmark disables by default, so that iterations reuse the prompt's cache; other settings were the defaults. For each KV dtype and point, `vllm bench latency` ran with output lengths 1 and 65 in three processes each (median of 15 iterations per process), and replicate i gives the decode step $(T_{65} - T_1)/64$; the 128K point uses 131,007 input tokens so that input and output fit the model's 131,072 positions (Table A10). vLLM selected FlashAttention for BF16 and FlashInfer for FP8 (`float8_e4m3fn`); the logs show no backend or dtype fallback, so the ratio includes the change of backend. The prediction is $T_{\text{pred,FP8}} = T_{\text{BF16}} - D_{\text{BF16}}/(2 BW_{\text{BF16}})$, with D_{BF16} the nominal BF16 K/V bytes per step at the mean attended context of the 64 differenced steps and $BW_{\text{BF16}} = 1,685$ GB/s fitted from the same BF16 baseline at $B = 1$, 32K and 128K. It assumes equal non-cache time for both dtypes, exactly half the bytes for FP8, and the $B = 1$ bandwidth at $B = 8$; the bytes are nominal, not measured DRAM traffic, and the bandwidth is a fitted parameter, not a hardware counter. Measured over predicted FP8 step time is 0.983–1.007. These vLLM timings compare only with vLLM's BF16, not with the harness of the main paper.

Throughput at fixed memory (synthetic cache, timing only). Seeded Gaussian K/V written through each method's store path replace the prefill; everything after cache construction is the timing protocol above. At $B = 1$, 32K, synthetic and real caches give step times within 0.05% for BF16 and KIVI-k4v4 (existing adapter), inside the preregistered 1% tolerance. At 128K the steady-state feasibility formula admits $B = 3$ for BF16 and $B = 9$ for KIVI-k4v4 under the 89.7 GB limit (Table A11); both ran, with allocator peaks below the predictions. These batch sizes lie outside the admitted geometries; only the geometry admission checks were bypassed, with no kernel or layout change. The measurement covers one decode step of the full model with BF16 weights resident and CUDA Graph replay, not prefill or request-level serving. KIVI-k4v4 delivers $0.84 \times$ BF16's tokens per second (process-pair range 0.841–0.843).

Deviations. The FP8 control exceeded its 3 h cap during installation and was rerun with the author's approval; FlashInfer's JIT build needed curand headers taken from the environment's own CUDA package, and because the bench-

Table A8. TurboQuant with 32 KV splits (post hoc; wall-clock; three processes per point; same-run BF16). S range: the nine ratios between BF16 and TurboQuant process medians. T/T_{BF16} : wall-clock; traced: GPU kernel time with 32 splits over BF16’s (Nsight Systems, Appendix D). The last three rows vary the split count for TQ-k3v4 at $B = 1, 128\text{K}$; as ported (4 splits) its S is 0.072 in the frozen scan.

Configuration	B	L	T_{BF16} (ms)	T (ms)	S	S range	T/T_{BF16}	Traced
TQ-4bit	1	32K	14.10	26.02	0.542	0.542–0.543	1.85	1.89
TQ-4bit	1	128K	22.26	68.20	0.326	0.326–0.327	3.06	3.16
TQ-4bit	8	32K	32.63	56.78	0.575	0.574–0.583	1.74	1.68
TQ-k3v4	1	32K	14.10	22.09	0.638	0.638–0.639	1.57	1.60
TQ-k3v4	1	128K	22.26	52.47	0.424	0.424–0.424	2.36	2.44
TQ-k3v4	8	32K	32.63	59.83	0.545	0.544–0.547	1.83	1.76
TQ-3bit	1	32K	14.10	23.44	0.601	0.601–0.602	1.66	1.69
TQ-3bit	1	128K	22.26	57.88	0.385	0.384–0.385	2.60	2.68
TQ-3bit	8	32K	32.63	64.09	0.509	0.507–0.510	1.96	1.89
TQ-k3v4, 8 splits	1	128K	22.26	159.61	0.139	0.139–0.140	7.17	—
TQ-k3v4, 16 splits	1	128K	22.26	85.94	0.259	0.259–0.260	3.86	—
TQ-k3v4, 64 splits	1	128K	22.26	37.00	0.602	0.601–0.602	1.66	—

Table A9. KIVI with the KV-head-grouped residual (post hoc; *diagnostic only*, because the adapter fails the 100-token greedy-identity gate; wall-clock; three processes per point; same-run BF16). S range: the nine ratios between BF16 and KIVI process medians. S existing: the frozen primary-scan ratio of the existing adapter (a different run; context only). S_{adj} : the post-hoc bound of Section 3.3, sealed before these runs and not refitted.

Configuration	B	L	T_{BF16} (ms)	T (ms)	S	S range	S existing	S_{adj}	$ S - S_{\text{adj}} $
KIVI-k4v4	1	4K	11.75	12.70	0.925	0.924–0.927	0.618	0.943	0.019
KIVI-k4v4	1	32K	14.09	18.18	0.775	0.772–0.775	0.575	0.786	0.011
KIVI-k4v4	1	128K	22.26	37.46	0.594	0.590–0.594	0.486	0.568	0.026
KIVI-k4v4	8	4K	14.26	15.11	0.944	0.944–0.944	0.678	0.949	0.005
KIVI-k4v4	8	24K	27.46	30.41	0.903	0.902–0.903	0.748	0.894	0.009
KIVI-k4v4	8	32K	32.62	37.14	0.878	0.878–0.884	0.756	0.878	0.000
KIVI-k2v2	1	4K	11.75	12.79	0.918	0.918–0.919	0.616	0.941	0.023
KIVI-k2v2	1	32K	14.09	18.62	0.757	0.749–0.757	0.565	0.767	0.010
KIVI-k2v2	1	128K	22.26	39.02	0.570	0.566–0.571	0.471	0.547	0.023
KIVI-k2v2	8	4K	14.26	14.72	0.969	0.968–0.969	0.690	0.987	0.018
KIVI-k2v2	8	24K	27.46	27.38	1.003	1.001–1.004	0.819	1.005	0.002
KIVI-k2v2	8	32K	32.62	32.47	1.005	1.004–1.009	0.841	1.001	0.003

Table A10. FP8 positive control in vLLM v0.25.1 (post hoc): de-code step $(T_{65} - T_1)/64$ in ms, median of three replicates; range over the nine replicate pairs. The prediction halves BF16’s nominal cache-path time on the same vLLM BF16 baseline; meas./pred. above 1 means FP8 is slower than predicted.

B	L	BF16	FP8	BF16/FP8	Range	FP8 pred.	Meas./pred.
1	32K	12.89	11.70	1.10	1.10–1.11	11.61	1.007
1	128K	20.53	15.17	1.35	1.35–1.36	15.44	0.983
8	32K	30.18	20.01	1.51	1.50–1.51	19.98	1.002

mark disables prefix caching by default, the rerun enabled it, again with the author’s approval. A planned KVQuant identity pass-through diagnostic was not run because its implementation estimate exceeded the preregistered 1 h.

Table A11. Throughput at fixed memory, 128K, synthetic cache (timing only; post hoc). Predicted: steady-state formula; allocator: PyTorch peak reserved bytes in the worker (excludes the CUDA context).

Configuration	B	Predicted (GB)	Allocator (GB)	Step (ms)	Tokens/s
BF16	3	74.1	67.8	43.24	69.4
KIVI-k4v4	9	83.7	65.0	154.09	58.4